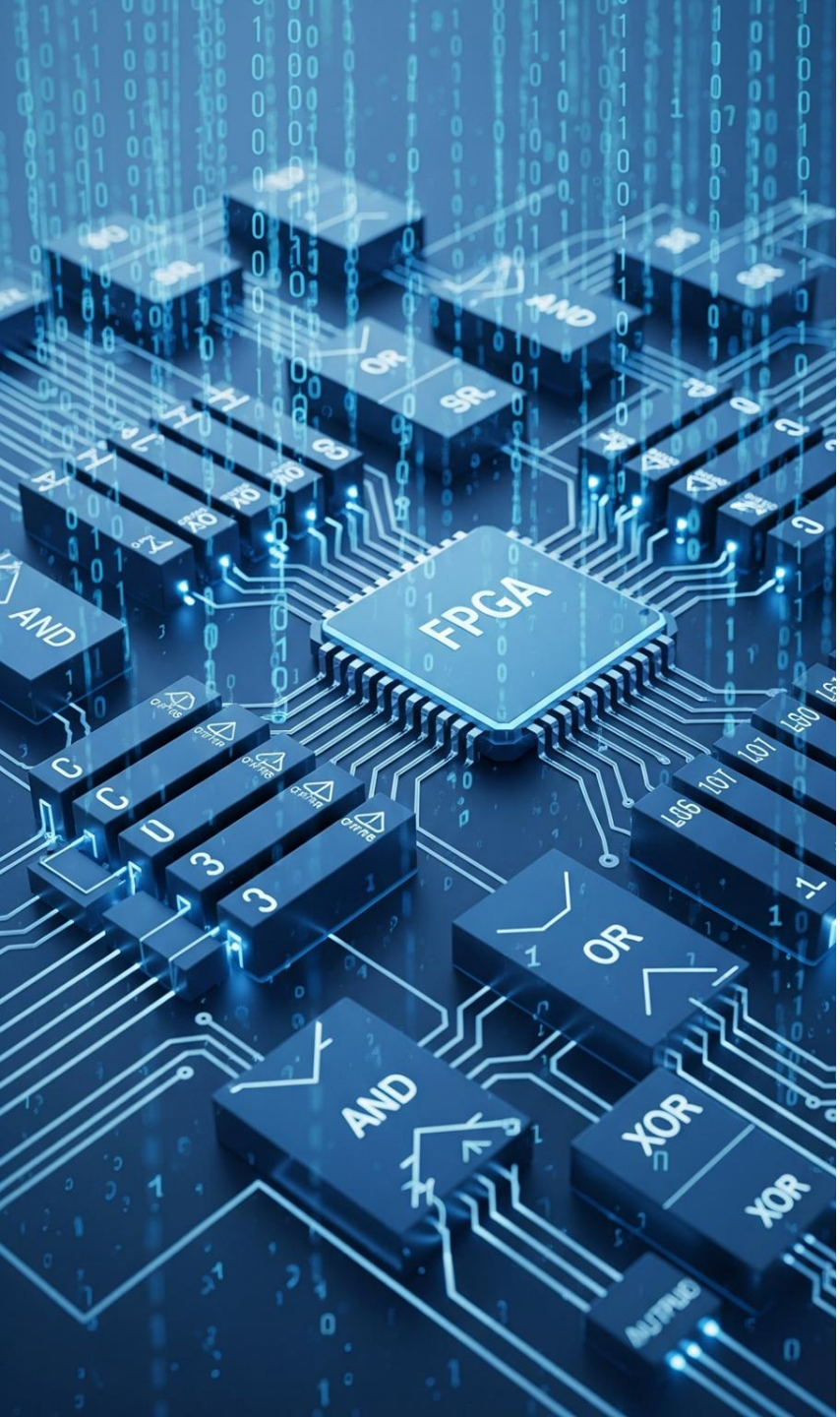


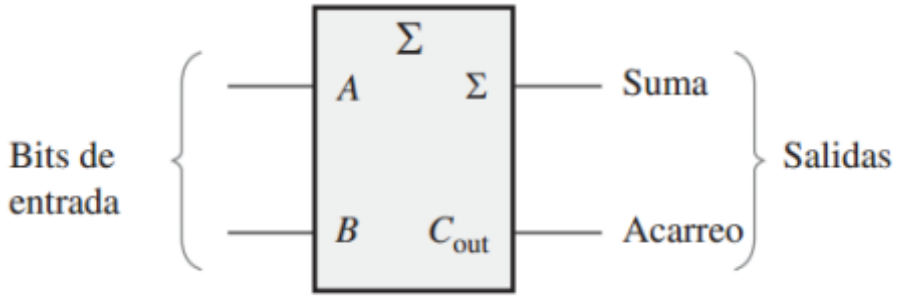
Técnicas Digitales

Lógica Combinacional
Sumador - restador

SUMADORES

El semi-sumador

Un semi-sumador suma dos bits y genera una suma y una salida de acarreo.



Reglas básicas de la suma binaria

$$0 + 0 = 0$$
$$0 + 1 = 1$$
$$1 + 0 = 1$$
$$1 + 1 = 10$$

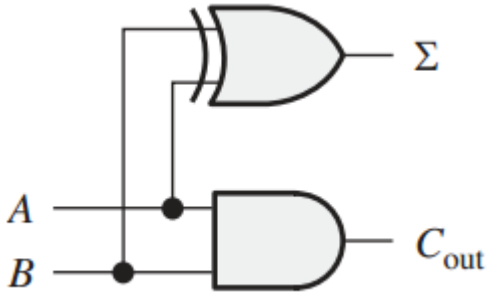
A	B	C_{OUT}	Σ
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

Σ = suma
C_{out} = acarreo de salida
A y B = variables de entrada (operandos)

$$C_{out} = AB$$

$$\Sigma = A \oplus B$$

Circuito lógico

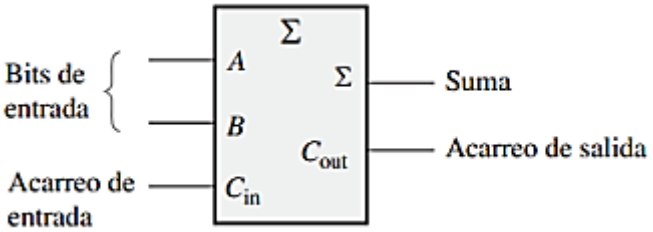


SUMADORES

¿Qué pasa si viene un acarreo de una suma anterior?

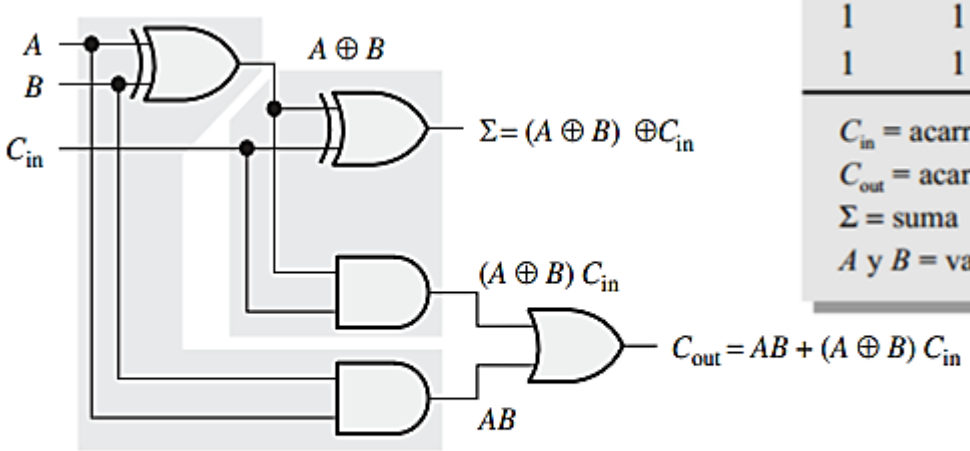
El sumador completo (Full adder)

Un sumador completo acepta dos bits de entrada y un acarreo de entrada, y genera una salida de suma y un acarreo de salida.



A	B	C_{in}	C_{OUT}	Σ
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

C_{in} = acarreo de entrada. Algunas veces se designa como *CI*.
C_{out} = acarreo de salida. Algunas veces se designa como *CO*.
Σ = suma
A y B = variables de entrada (operandos)



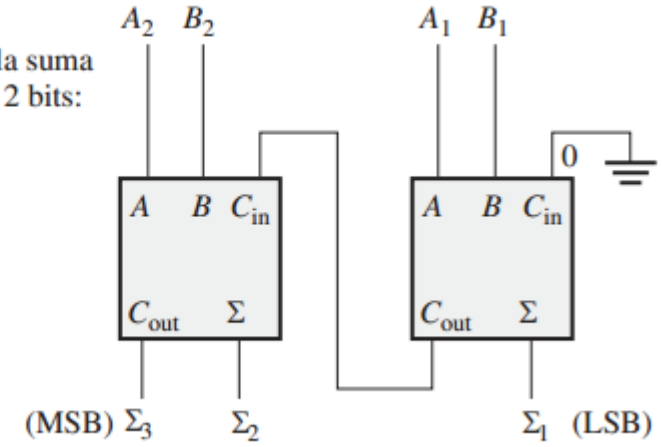
Circuito lógico de un sumador completo (cada semi-sumador se representa por un área sombreada)

SUMADORES BINARIOS EN PARALELO

Para formar un sumador binario en paralelo se conectan dos o más sumadores completos.

Formato general de la suma de dos números de 2 bits:

$$\begin{array}{r} A_2A_1 \\ +\,B_2B_1 \\ \hline \Sigma_3\Sigma_2\Sigma_1 \end{array}$$



Sumador en paralelo de cuatro bits **(Nibble)**

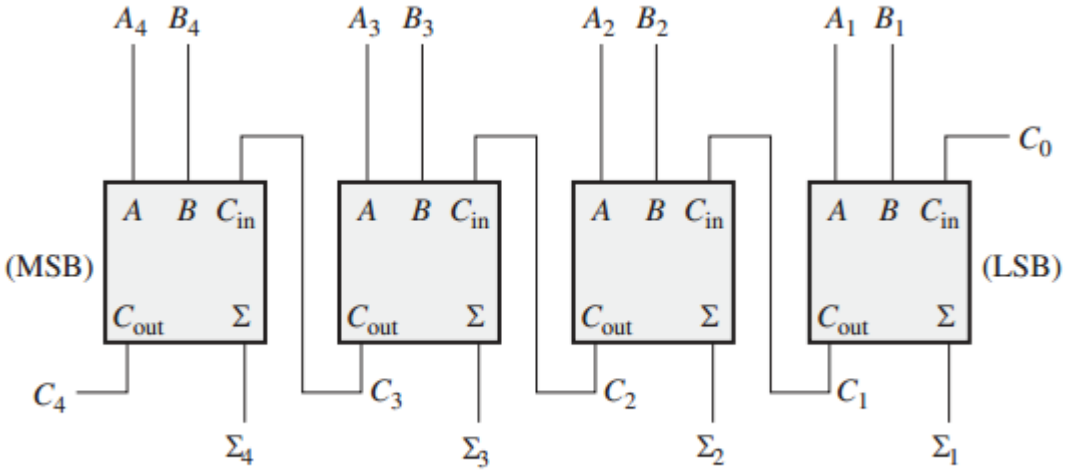
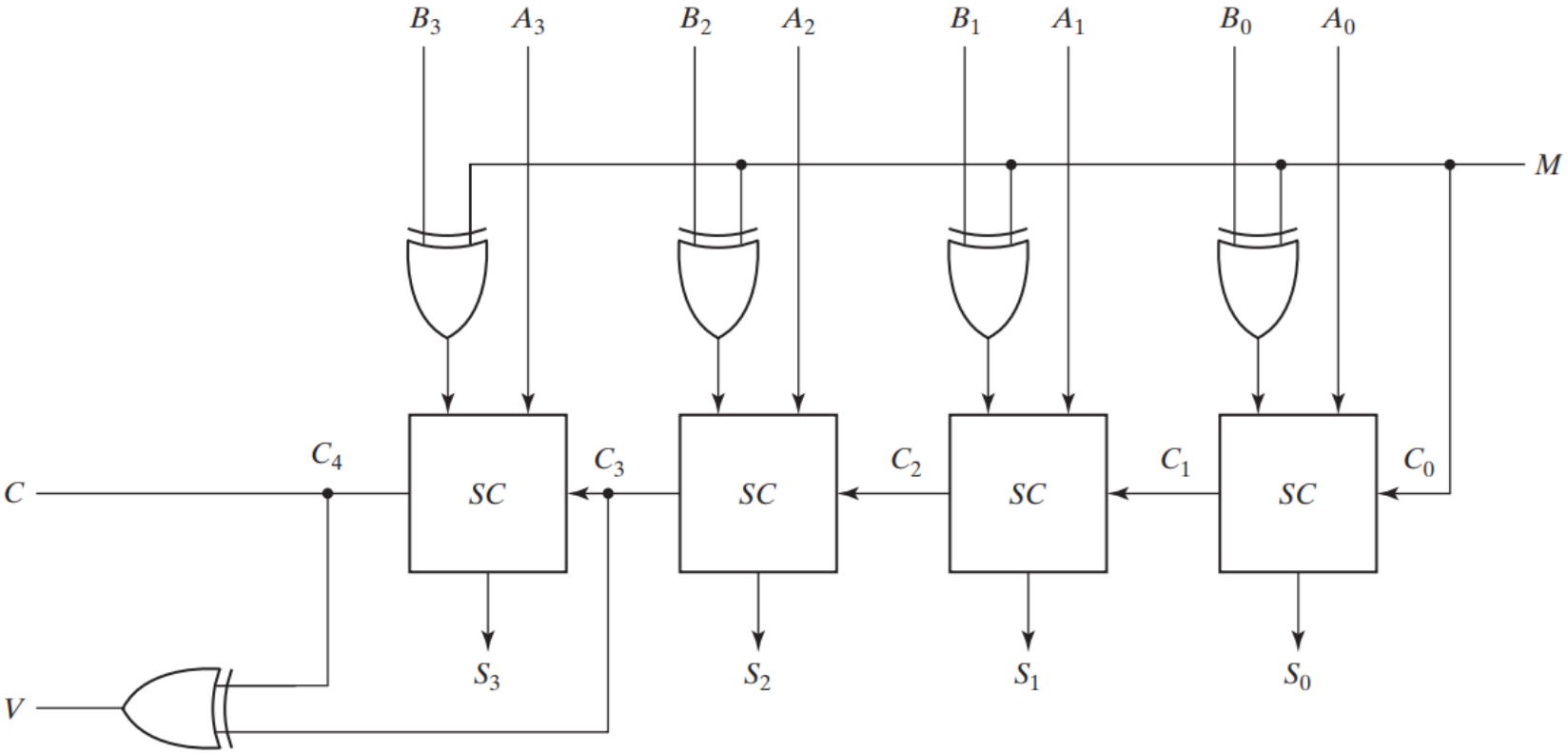


Diagrama de bloques

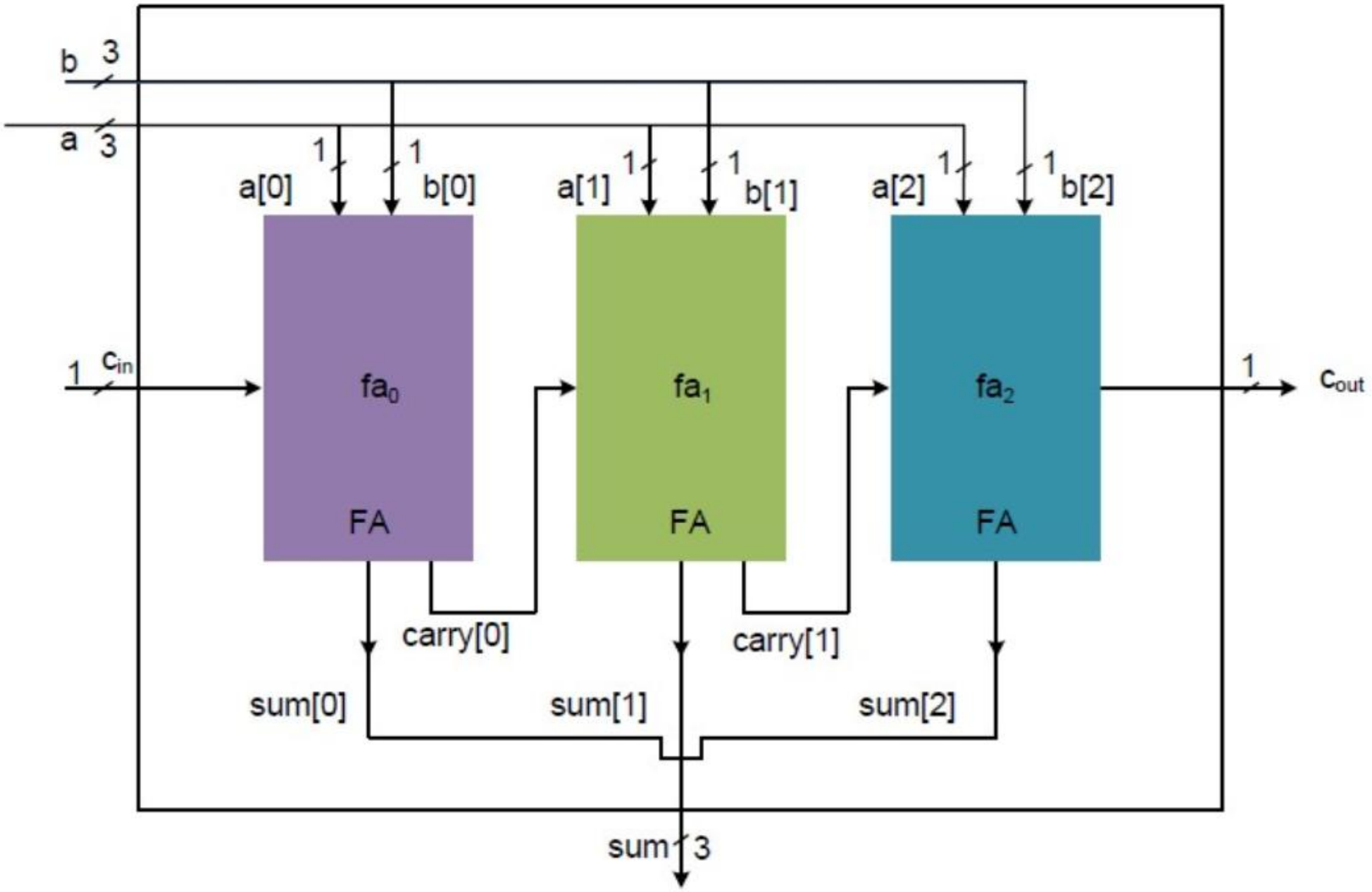
SUMADOR - RESTADOR BINARIO



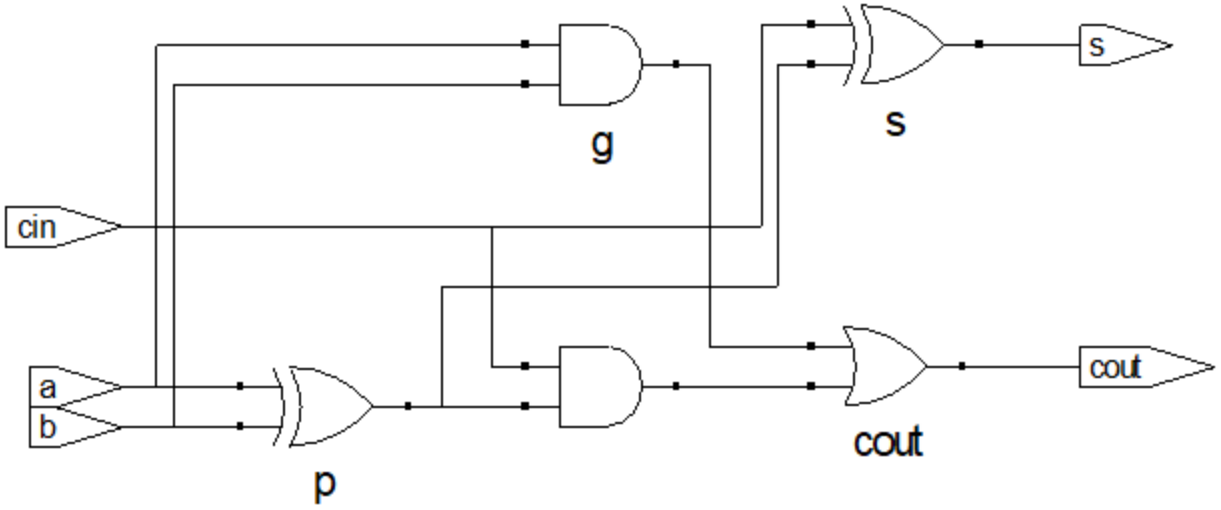
OVERFLOW

acarreos:	0	1		acarreos	1	0
+70	0	1000110		−70	1	0111010
+80	0	1010000		−80	1	0110000
+150	1	0010110		−150	0	1101010

Sumador de 2 números de 3 bits



FULL ADDER en Verilog




```
module fulladder(input  a, b, cin, output s, cout);
  wire p, g;        // internal nodes

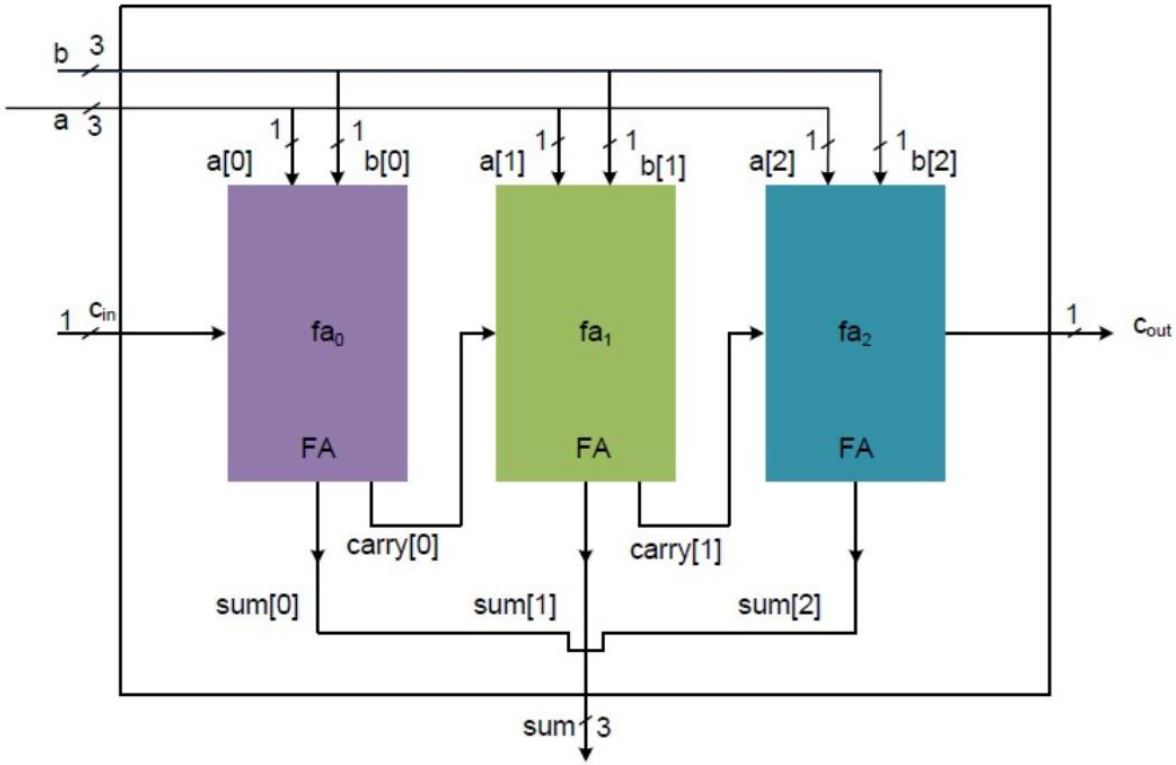
  assign p = a ^ b;
  assign g = a & b;
  assign s = p ^ cin;
  assign cout = g | (p & cin);

endmodule
```

Instanciación sumador de 2 números de 3 bits

```
module fulladder 1
    input [2:0] A,
    input [2:0] B,
    input CIN,
    output COUT,
    output [3:0] sum
    );

wire carry [1:0];
```




```
fulladder FA0 (.a(A[0]), .b(B[0]), .cin(CIN), .s(suma[0]), .cout(carry[0]) );

fulladder FA1 (.a(A[1]), .b(B[1]), .cin(carry[0]), s(suma[1]), .cout(carry[1]) );

fulladder FA2 (.a(A[2]), .b(B[2]),  .cin(carry[1]), .s(suma[2]), .cout(COUT) );

endmodule
```

Multiplexor 4 a 1 (case)

```
module mux4_1 (
    input [1:0] sel,      // Señal de selección de 2 bits
    input in0,            // Entrada 0
    input in1,            // Entrada 1
    input in2,            // Entrada 2
    input in3,            // Entrada 3
    output reg out        // Salida
);


always @(*) begin
    case (sel)
        2'b00: out = in0;  // Si sel = 00, salida = in0
        2'b01: out = in1;  // Si sel = 01, salida = in1
        2'b10: out = in2;  // Si sel = 10, salida = in2
        2'b11: out = in3;  // Si sel = 11, salida = in3
        default: out = 1'b0; // Por defecto (buena práctica)
    endcase
end

endmodule
```



Multiplexor 4 a 1 (if-else)

```
module mux4_1 (
    input [1:0] sel,      // Señal de selección de 2 bits
    input in0,            // Entrada 0
    input in1,            // Entrada 1
    input in2,            // Entrada 2
    input in3,            // Entrada 3
    output reg out        // Salida
);

  always @(*) begin
      if (sel == 2'b00)
          out = in0;
      else if (sel == 2'b01)
          out = in1;
      else if (sel == 2'b10)
          out = in2;
      else
          out = in3;
  end

  endmodule
```



Multiplexor 4 a 1 (operador ternario)

```
module mux4_1 (
    input [1:0] sel,      // Señal de selección de 2 bits
    input in0,            // Entrada 0
    input in1,            // Entrada 1
    input in2,            // Entrada 2
    input in3,            // Entrada 3
    output reg out        // Salida
);

  assign    out = (sel == 2'b00) ? in0 :
                  (sel == 2'b01) ? in1 :
                  (sel == 2'b10) ? in2 :
                   in3;

  endmodule
```